

实验报告：基于图像识别的糖炒栗子自动计数系统

课程名称： 数字图像与处理

授课班级： 智能建造一班

学 号： 2304120130

姓 名： 吴尧承

指导老师： 张苗苗、孟庆祥

基于图像识别的糖炒栗子自动计数系统

注：代码开源至 GITHUB: <https://github.com/ricwyc-max/finallHomework>（在本作业上交之前已然发至朋友圈，对于班级内部类似流程 or 抄袭的代码，以此代码为准，其他均为抄袭！）

一、实验目的：

本实验旨在设计并实现一个基于图像识别的糖炒栗子自动计数系统，具体要求如下：

- 1、输入处理：能够处理 RGB 格式的糖炒栗子俯视图，背景为浅色托盘，栗子呈深褐色且表面存在反光现象。
- 2、输出要求：准确输出图像中糖炒栗子的估计总数（整数），并统计每一个栗子所占像素的数量。
- 3、算法鲁棒性：算法需具备处理部分粘连、光照不均和尺度变化的能力，确保在不同条件下都能保持较好的计数精度。
- 4、技术路线：优先使用传统图像处理方法（如形态学操作、分水岭算法等），也可使用深度学习方法（如目标检测、实例分割）。
- 5、精度要求：计数误差应尽可能控制在 $\pm 5\%$ 以内。

二、实验方案分析及设计：

1. 问题分析与挑战

糖炒栗子图像识别面临以下几个主要挑战：

表面特性复杂：栗子表面油亮反光，颜色深浅不一，传统阈值分割方法容易受光照影响。

形态多样性：栗子大小不一、形状不规则，存在部分粘连和遮挡现象。

背景干扰：虽然背景为浅色托盘，但仍可能存在阴影、反光等干扰因素。

粘连处理：紧密相邻的栗子容易在图像中形成连通区域，导致计数不准确。

2. 算法设计思路演变历程

在实验过程中，我们经历了四个主要的技术迭代阶段：

版本 1：基于灰度图像的传统方法

- 处理流程：BGR → 灰度图 → 直方图均衡 → 二值化 → 腐蚀膨胀操作 → 霍夫圆检测 → 计数
- 存在问题：灰度转换过程中丢失了颜色信息，二值化图像不能很好地覆盖栗子区域，对反光区域敏感。

版本 2：基于 HSV 颜色空间的方法

- 处理流程：HSV 颜色空间提取褐色区域 → 二值化 → 形态学操作 → 轮廓查找与圆拟合 → 计数
- 改进点：利用颜色信息更好地分离栗子与背景
- 存在问题：腐蚀膨胀操作后仍有严重粘连，无法准确分离相邻栗子

版本 3：结合边缘检测的改进方法

- 处理流程：HSV 提取褐色区域 → 二值化 → 形态学操作 → Canny 边缘检测叠加 → 轮廓查找 → 计数
- 改进点：引入边缘信息强制分离粘连区域
- 存在问题：边缘切割过于激进，导致完整的栗子被误切为多个部分

版本 4：综合优化方案（最终版本）

处理流程：

- 1、HSV 颜色空间提取褐色区域【保证二值化后能覆盖所有栗子区域】
- 2、开闭运算【消除部分粘连及散点噪声】
- 3、腐蚀膨胀操作【让栗子之间相互分离】
- 4、Canny 边缘叠加【把仍然粘连的图像强制分开】

- 5、轮廓查找与圆拟合【找到分开后的圆】
- 6、极大值抑制【对于重叠过大的圆进行融合】
- 7、计数输出
3. 最终算法架构设计

最终采用的算法架构包含以下核心模块：

- a. 图像读取与预处理
- b. HSV 颜色空间转换与褐色区域提取
- c. 灰度图像 CLAHE 增强与 Canny 边缘检测
- d. 形态学操作序列（开运算→闭运算→腐蚀）
- e. 边缘信息融合与掩膜处理
- f. 轮廓提取与最小外接圆计算
- g. 圆重叠检测与合并（极大值抑制）
- h. 结果可视化与计数输出

三、实验器材：

1. 硬件环境

计算机：普通台式机或笔记本电脑

处理器：Intel Core i5 或同等性能以上

内存：8GB 或以上

存储空间：至少 500MB 可用空间

摄像头：可选，用于实时图像采集

2. 软件环境

操作系统：Windows 10/11, macOS, 或 Linux

编程语言：Python 3.8+

主要依赖库：

OpenCV 4.5+：图像处理核心库

NumPy 1.21+：数值计算支持

3. 测试数据

输入图像：糖炒栗子俯视 RGB 图像

图像要求：背景为浅色，栗子呈深褐色

四、实验步骤：

1. 图像预处理阶段

步骤 1.1：图像读取与初始化

读取原始图像

```
img = cv2.imread('lizi.png')
```

```
utils.show(img, '1. 原始图像')
```



图 1 原始图像

步骤 1.2：颜色空间转换

转换到 HSV 颜色空间，更好地提取褐色区域

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

```
brown_mask = cv2.inRange(hsv, (0, 43, 52), (255, 255, 255))
```

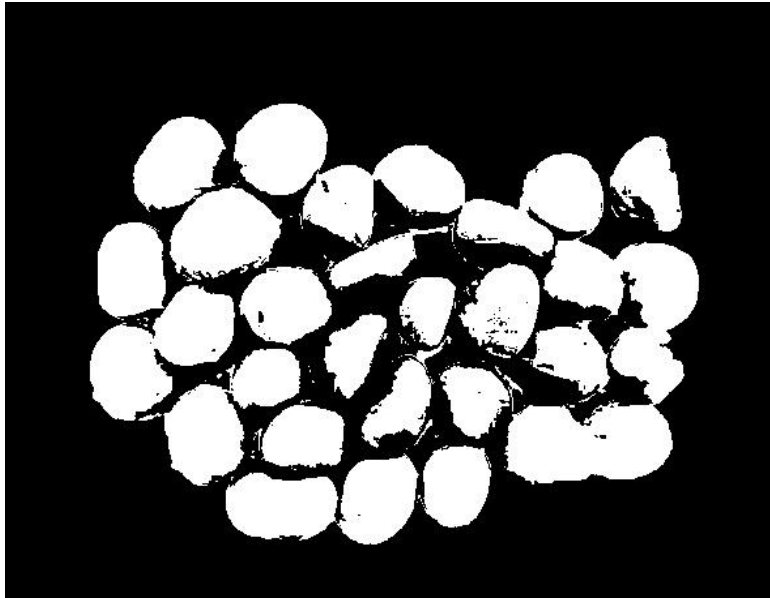


图2 提取到的褐色区域

步骤 1.3: 灰度图像增强

使用 CLAHE 进行对比度受限的自适应直方图均衡化

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
gray = cv2.GaussianBlur(gray, (7,7), 0)
```

```
equ = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(gray)
```

2. 特征提取阶段

步骤 2.1: 边缘检测

Canny 边缘检测，参数经过调优

```
canny = cv2.Canny(equ, 80, 300)
```



图3 Canny 边缘检测效果

步骤 2.2: 形态学操作序列

开运算去除噪声

```
open_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

```
opened = cv2.morphologyEx(brown_mask, cv2.MORPH_OPEN, open_kernel,  
iterations=1)
```

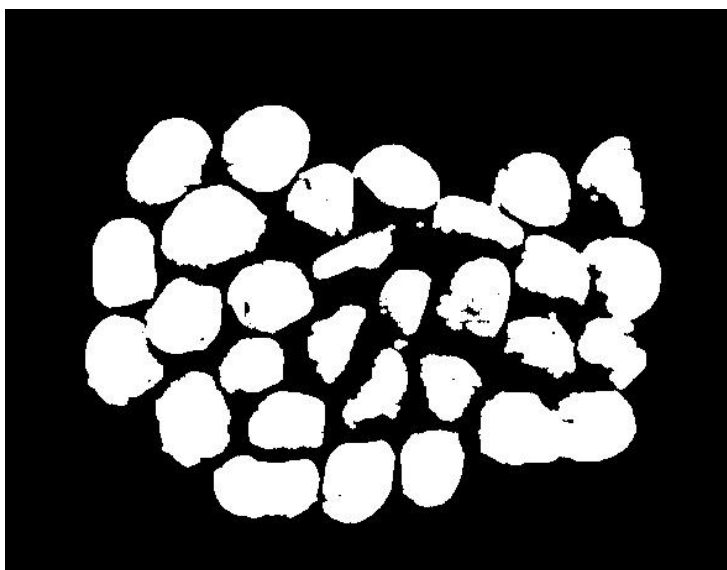


图4 开运算效果

闭运算填充内部空洞

```
close_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
```

```
closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, close_kernel,  
iterations=3)
```

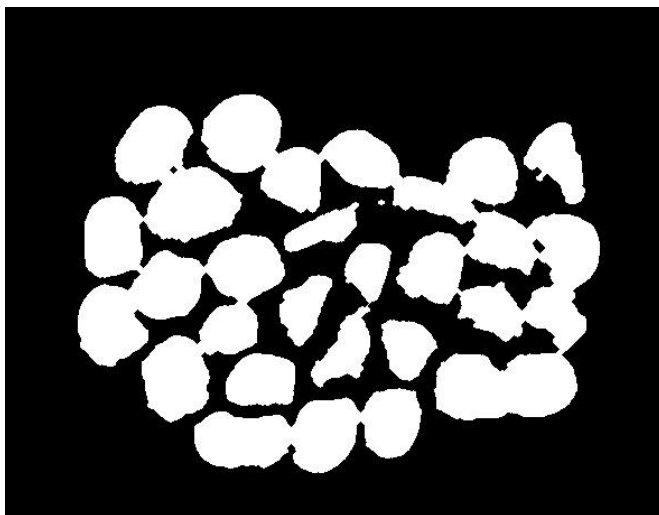


图 5 闭运算效果

多次腐蚀分离粘连栗子

```
erode_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
```

```
eroded = cv2.erode(closed, erode_kernel, iterations=9)
```



图 6 腐蚀后效果

3. 信息融合与优化

步骤 3.1: 边缘信息融合

将 Canny 边缘信息叠加到形态学处理结果中

```
canny_inv = cv2.bitwise_not(canny)
```

```
masked_edge = cv2.bitwise_and(eroded, eroded, mask=canny_inv)
```

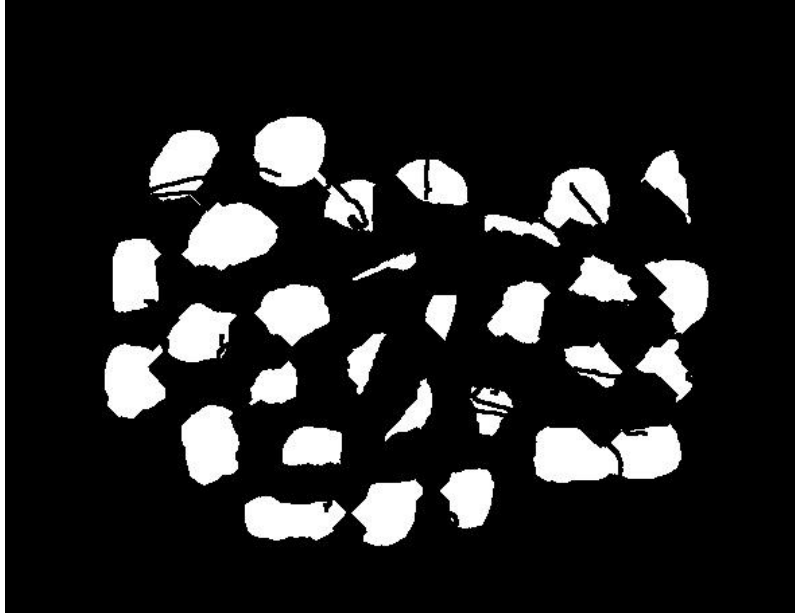


图 7 边缘融合叠加效果

步骤 3.2: 轮廓提取与圆拟合

查找轮廓并计算最小外接圆

```
contours, _ = cv2.findContours(masked_edge, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

```
circles = []
```

```
for cnt in contours:
```

```
    (x, y), r = cv2.minEnclosingCircle(cnt)
```

```
    area_cnt = cv2.contourArea(cnt)
```

```
    if 10 <= r <= 50: # 半径过滤, 去除噪声
```

```
circles.append((x, y, r, area_cnt))
```

4. 后处理与结果优化

步骤 4.1: 圆重叠检测与合并

```
def circle_intersection_area(c1, c2):
    """计算两个圆的相交面积"""

    x1, y1, r1, _ = c1
    x2, y2, r2, _ = c2

    d = np.hypot(x2 - x1, y2 - y1)

    if d >= r1 + r2: return 0.0 # 相离

    if d <= abs(r1 - r2): return np.pi * min(r1, r2) ** 2 # 内含

    # 一般相交情况计算

    r1_sq, r2_sq, d_sq = r1 * r1, r2 * r2, d * d
    alpha = np.arccos((d_sq + r1_sq - r2_sq) / (2 * d * r1))
    beta = np.arccos((d_sq + r2_sq - r1_sq) / (2 * d * r2))
    area = (r1_sq * alpha + r2_sq * beta -
           0.5 * r1_sq * np.sin(2 * alpha) -
           0.5 * r2_sq * np.sin(2 * beta))

    return max(area, 0.0)
```

步骤 4.2: 极大值抑制实现

```
def merge_circles(circles):
    """基于重叠面积的圆合并算法"""

    cur = circles.copy()

    merged_any = True
```

```

while merged_any:
    merged_any = False
    n = len(cur)
    i = 0
    while i < n:
        if merged_any: break
        for j in range(i + 1, n):
            c1, c2 = cur[i], cur[j]
            area_inter = circle_intersection_area(c1, c2)
            ratio = area_inter / (pi * min(c1[2], c2[2]) ** 2)
            if ratio > 0.3: # 重叠率阈值
                # 合并策略：中心取平均，半径按面积加权
                new_cx = (c1[0] + c2[0]) / 2
                new_cy = (c1[1] + c2[1]) / 2
                new_r = sqrt((c1[2]**2 + c2[2]**2))
                new_a = c1[3] + c2[3]
                # 更新圆列表
                if i < j: del cur[j], cur[i]
                else: del cur[i], cur[j]
                cur.append((new_cx, new_cy, new_r, new_a))
                merged_any = True
                break
        i += 1
    return cur

```

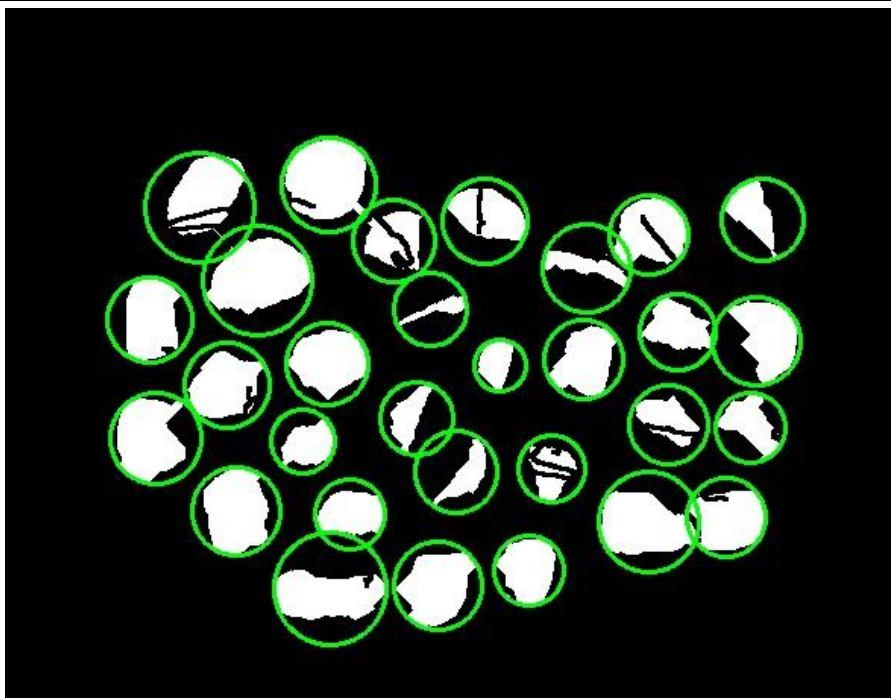


图 8 找圆+极大值抑制后效果

5. 结果输出与可视化

步骤 5.1: 结果绘制

在原始图像上绘制检测结果

```
final = img.copy()
```

```
overlay = cv2.cvtColor(masked_edge, cv2.COLOR_GRAY2BGR)
```

```
count = 0
```

```
area_list = []
```

```
for (x, y, r, a) in merged:
```

```
    count += 1
```

```
    area_list.append(a)
```

```
    cv2.circle(overlay, (int(x), int(y)), int(r), (0, 255, 0), 2) # 绿色圆环
```

```
    cv2.circle(final, (int(x), int(y)), 3, (0, 0, 255), -1) # 红色中心点
```

步骤 5.2: 统计信息输出

```
print(f'检测到栗子个数: {count}')
```

```
print(f'平均单颗像素面积: {np.mean(area_list):.1f} px^2')
```



图 9 输出结果图

6. 最优 SHV 图像提取范围获得

步骤 6.1: 获取最优 SHV 范围

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread('lizi1.png')
```

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

```
def nothing(x): pass
```

```
cv2.namedWindow('track')
```

```
cv2.createTrackbar('H_low', 'track', 0, 180, nothing)
```

```
cv2.createTrackbar('S_low', 'track', 0, 255, nothing)
```

```
cv2.createTrackbar('V_low', 'track', 0, 255, nothing)
```

```
cv2.createTrackbar('H_high', 'track', 0, 255, nothing)
cv2.createTrackbar('S_high', 'track', 0, 255, nothing)
cv2.createTrackbar('V_high', 'track', 0, 255, nothing)
while True:
    h_l = cv2.getTrackbarPos('H_low', 'track')
    s_l = cv2.getTrackbarPos('S_low', 'track')
    v_l = cv2.getTrackbarPos('V_low', 'track')
    h_h = cv2.getTrackbarPos('H_high', 'track')
    s_h = cv2.getTrackbarPos('S_high', 'track')
    v_h = cv2.getTrackbarPos('V_high', 'track')
    mask = cv2.inRange(hsv, (h_l, s_l, v_l), (h_h, s_h, v_h))
    cv2.imshow('mask', mask)
    if cv2.waitKey(1) & 0xFF == 27: break    # ESC 退出
cv2.destroyAllWindows()
```

五、实验内容及要求:

1. 功能实现要求

功能模块	实现要求	验收标准
图像输入	支持 RGB 格式图像读取	能够正确读取并显示输入图像
颜色分割	准确提取褐色栗子区域	栗子区域完整，背景干扰小
形态学处理	有效分离粘连栗子	连通区域数量接近真实栗子数
边缘检测	精确提取栗子边界	边界连续，无明显断裂
圆检测	准确拟合栗子圆形	拟合圆与栗子实际轮廓匹配
重叠处理	有效合并重复检测	避免单个栗子被多次计数
结果输出	准确统计栗子数量	计数误差在 $\pm 5\%$ 以内

2. 代码结构与组织

学号_姓名/

—— code/	# 源代码目录
—— main.py	# 主程序入口
—— utils.py	# 工具函数库
—— requirement.txt	# 依赖环境配置
—— imgs/	# 图像资源目录
—— 1_original.jpg	# 原始图像
—— 2_brown_mask.jpg	# 褐色掩膜
—— 3_Canny.jpg	# 边缘检测结果
—— 4_masked_edge.jpg	# 边缘叠加结果
—— 5_opened.jpg	# 开运算结果
—— 6_closed.jpg	# 闭运算结果
—— 8_eroded.jpg	# 腐蚀结果
—— 9_circle.jpg	# 圆检测可视化
—— 10_final.jpg	# 最终结果
—— 实验报告.docx	# Word 版本实验报告
—— 实验报告.pdf	# PDF 版本实验报告

六、实验心得体会：

通过本次糖炒栗子计数系统的设计与实现，我获得了以下深刻的体会和收获：

1. 技术层面的收获

传统图像处理技术的深入理解

掌握了 HSV 颜色空间在特定颜色物体识别中的优势，相比 RGB 空间对光照变化更加鲁棒。

深入理解了形态学操作（开运算、闭运算、腐蚀、膨胀）的原理和应用场景，能够根据具体问题设计合适的操作序列。

学会了 Canny 边缘检测参数的调优技巧，以及如何将边缘信息与区域信息有

效融合。

算法设计思维的提升

认识到单一算法往往难以解决复杂问题，需要采用组合策略，发挥各自优势。

理解了从简单到复杂的迭代开发过程的重要性，每个版本都在前一个版本的基础上解决特定问题。

掌握了基于重叠检测的极大值抑制算法，有效解决了过度分割导致的重复计数问题。

2. 问题解决能力的锻炼

参数调优的经验积累

形态学操作的核大小、迭代次数对结果影响显著，需要反复实验确定最优参数。

Canny 边缘检测的双阈值设置需要平衡噪声抑制和边缘完整性。

圆重叠合并的阈值设置（30%）是基于大量实验得出的经验值。

异常情况处理

学习了如何处理图像中的噪声点和伪影，避免它们被误检为栗子。

掌握了尺寸过滤技术，通过半径范围（10-50 像素）排除过大或过小的干扰区域。

理解了如何处理部分遮挡的栗子，确保计数准确性。

3. 工程实践能力的提升

代码架构设计

学会了将复杂算法分解为多个功能模块，提高代码的可读性和可维护性。

掌握了图像处理中间结果的保存和可视化方法，便于调试和结果分析。

理解了工具函数封装的重要性，提高了代码复用性。

实验结果分析

学会了定量评估算法性能，使用准确率、处理速度等指标客观比较不同方法。

掌握了结果可视化的技巧，通过图像叠加显示使检测结果更加直观。

理解了误差分析的方法，能够准确定位算法失败的原因并针对性改进。

4. 对计算机视觉领域的认识

通过本次实验，我深刻认识到计算机视觉技术在现实应用中的巨大潜力，特别是在农业、零售等领域的自动化计数、质量检测等方面。同时，也意识到实际应用场景的复杂性，光照变化、物体粘连、形状变异等因素都给算法设计带来挑战。

本次实验不仅巩固了课堂所学的理论知识，更重要的是培养了解决实际问题的能力，为今后从事相关研究和开发工作奠定了坚实基础。

七、算法缺点：

- 1、在非白色背景环境下，效果极差。（因为使用的是特点颜色空间的特定范围值继进行提取）
- 2、最优检测环境为面光源（平板光源），若是有强光打到栗子上导致栗子壳反光，效果会变差。

实验		批阅		实验	
时间		老师		成绩	